Weather Advisor in the Cloud: Two Tiers, a Free AI Model and a PostgreSQL Database

Course: ITMO 444/544 Cloud Computing / Cloud Security
Level: Advanced
Estimated Time: 170 to 200 minutes
Prerequisites: The previous lab completed. Undergraduate students continue from `lab04` (ALB and Auto Scaling Groups, `.env`, the `testing/` folder). Graduate students continue from `lab05` (the same, plus the Python package and pytest tests). You will copy that folder to a new `lab06` folder in Part 0 and work only there.

---

1. Learning Objectives

By the end of this lab you will be able to:

1. Call two public web APIs from a cloud service: the **National Weather Service API** (forecasts) and a free **zip code lookup API** (zip code to latitude and longitude).
2. Build a **two-tier application** behind one Application Load Balancer: a *weather service* that fetches forecast data, and an *advisor service* that asks a free **Google Gemini** model for a simple clothing suggestion.
3. Keep secrets (a third-party **API key** and a **database password**) out of your code, your `.env` file and your launch templates by storing them in **SSM Parameter Store** as SecureStrings, each readable by only one instance role.
4. Prove with IAM's policy simulator that **each role can read only its own secret**.
5. Make a service **degrade gracefully**: when the AI model fails or is rate limited, the advisor still answers with a simple rule-based suggestion that is clearly labelled.
6. Treat AI output as **untrusted input** and escape it before it reaches a web page.
7. Explain what happens to the advisor when the weather service it depends on fails, and measure how long recovery takes.
8. Create a **free-tier PostgreSQL database on Amazon RDS** that is private, encrypted, single-zone and has **automated backups turned off**, and use it as both a **cache** and a **history table**.
9. Delete the database completely when testing is done, with **no snapshot and no backup left behind**, and prove it.
10. Stay inside the free tier, and say exactly what data you are sending to a free AI service.

---

2. Why This Matters

Real applications are rarely one program. They are several small services that depend on each other and on outside APIs that you do not control. The weather API can be slow. The AI service can reject your key or tell you to slow down. One of your own tiers can disappear. Each of these needs a deliberate answer in the design.

This lab gives you a small, safe place to practice three habits that every cloud engineer needs.

- **Dependencies fail, so plan the failure.** Timeouts, one retry, clear error codes, and a fallback answer.
- **Secrets stay out of files.** The Gemini key is the only secret in the stack, and exactly one role may read exactly one parameter.
- **Third-party AI is a data-sharing decision.** You choose what goes into the prompt, and you treat what comes back as untrusted.
- **Stored data has a lifecycle.** You decide what is stored, who can reach it, whether it is backed up, and exactly how it is destroyed. A database you forget about is the most common source of surprise cloud bills.

---

3. A Note for Graduate Students (ITMO 544)

> **Graduate students must do the lab work in Python, not in the bash scripts.**
>
> Undergraduate students (ITMO 444) build this lab with the AWS CLI and the bash scripts printed below. Graduate students start from `lab05`, which already contains the boto3 package from the Python lab, and must write the Python equivalent of every script in this lab. The Node.js apps are the same for everyone, because they are the application, not the infrastructure.
>
> Graduate students must:
>
> 1. Read the bash scripts in Part E as the **specification** of what to build, then write Python that creates and deletes the same resources: a `create_database.py`, `delete_database.py`, `create_weather_stack.py` and `delete_weather_stack.py`, built on your `stack/` package from `lab05`.
> 2. Write **pytest tests** for the new code. Unit tests use `moto`. Integration tests run against your real stack and are marked `integration`. The required tests are listed in the **Graduate Checklist** in Section 16.2.
> 3. Run the experiments in Part H with Python where a Python version makes sense. Plain `aws` commands are fine for **looking** at things, but every resource you **create or delete** must come from your Python code.
> 4. Submit the **Graduate Checklist** (Section 16.2) in addition to the shared items.
>
> Undergraduate students may also use Python, but it is not required.

New boto3 calls in this lab. You already know the EC2, ELB, Auto Scaling and IAM calls from lab05. These are the new ones.

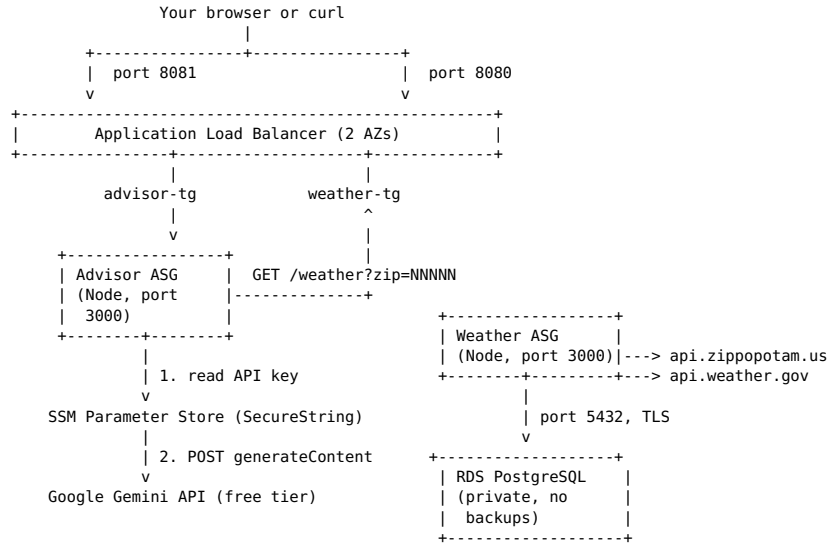
Task	AWS CLI	boto3
Store the key	`aws ssm put-parameter --type SecureString`	`ssm.put_parameter(Name=, Type="SecureString", Value=, Overwrite=True)`
Check it exists	`aws ssm get-parameter --name`	`ssm.get_parameter(Name=)` and catch `ssm.exceptions.ParameterNotFound`

Inline role policy	`aws iam put-role-policy`	`iam.put_role_policy(RoleName=, PolicyName=, PolicyDocument=json.dumps(doc))`
Remove it	`aws iam delete-role-policy`	`iam.delete_role_policy(RoleName=, PolicyName=)`
Prove a permission	`aws iam simulate-principal-policy`	`iam.simulate_principal_policy(PolicySourceArn=, ActionNames=[...], ResourceArns=[...])`
Wait for the ALB	`aws elbv2 wait load-balancer-available`	`elbv2.get_waiter("load_balancer_available").wait(LoadBalancerArns=[...])`
DB subnet group	`aws rds create-db-subnet-group`	`rds.create_db_subnet_group(DBSubnetGroupName=, DBSubnetGroupDescription=, SubnetIds=[...])`
Create the database	`aws rds create-db-instance`	`rds.create_db_instance(DBInstanceIdentifier=, DBInstanceClass="db.t4g.micro", Engine="postgres", AllocatedStorage=20, BackupRetentionPeriod=0, MultiAZ=False, PubliclyAccessible=False, StorageEncrypted=True, ...)`
Wait for it	`aws rds wait db-instance-available`	`rds.get_waiter("db_instance_available").wait(DBInstanceIdentifier=)`
Delete it with no leftovers	`aws rds delete-db-instance --skip-final-snapshot --delete-automated-backups`	`rds.delete_db_instance(DBInstanceIdentifier=, SkipFinalSnapshot=True, DeleteAutomatedBackups=True)`
Prove no snapshots remain	`aws rds describe-db-snapshots`	`rds.describe_db_snapshots()["DBSnapshots"]` and `rds.describe_db_instance_automated_backups()`

Three cautions apply to Python as much as bash. First, a new IAM role needs a short pause before EC2 will accept it, so keep a retry or a wait. Second, never put the database password in a variable that you log or print, and generate it with `secrets.token_hex(16)`. Third, moto implements most of these services but not every parameter. When a unit test cannot express something, test the function that **builds the request** (the policy document, the user-data text) and leave the full proof for an integration test.

---

4. Architecture Overview



Two details are worth noticing before you build anything.

- **The advisor calls the weather service through the load balancer**, not directly. That keeps the two tiers loosely coupled, and it means a replaced weather instance never changes any address the advisor knows about.
- **The ALB is created first in this lab.** The advisor's launch template needs the load balancer's DNS name, so the order of the create script is different from lab04.
- **Only the weather tier talks to the database.** The weather service saves every forecast it fetches and reuses a recent one for the same zip code. The advisor never touches the database, and each tier's role can read exactly one secret: the weather role reads the database password, and the advisor role reads the Gemini key.

---

5. Free Tier and Cost Guide (Read Before You Build)

Item	Free or paid	What this lab does
National Weather Service API	Free, no key	Needs only a `User-Agent` header
Zip code API (api.zippopotam.us)	Free, no key	Used to turn a zip code into coordinates
Google Gemini API, free tier	Free, with rate limits and a daily quota	One short request per advice. Limits change, so check the current ones in Google AI Studio
SSM Parameter Store, **standard** SecureString	Free for standard parameters	Two parameters (Gemini key, database password), encrypted with the default AWS managed

		key
Amazon RDS for PostgreSQL, `db.t4g.micro`, single-AZ, 20 GB	Free plan (accounts created after July 15, 2025): covered by your credits for up to 6 months. Legacy accounts: the 12-month RDS offer, which I believe is 750 instance hours and 20 GB a month (check the offer terms for your account)	One instance for a few hours, then deleted
RDS **Multi-AZ**, a larger instance class, a public address	Not free	**Not used.** Do not change these settings
RDS **backups**	Automated backups and snapshots use backup storage	**Turned off** (retention 0). Nothing is stored, and nothing is left behind when you delete
Application Load Balancer, EC2 `t3.micro` (or your free-tier type), Auto Scaling	See lab04	Two instances plus one ALB for a few hours

> ⚠ Free tier rules differ by account age. Accounts created on or after July 15, 2025 use a credit-based free plan, and older accounts use the legacy 12-month model plus always-free limits. This lab runs two small instances, a load balancer and a database, all billed by the hour once your free allowance or credits run out, so **build, test, and then run the cleanup step the same day**. A database left running for a month is the classic surprise bill. Check **Billing and Cost Management** afterward.

> ⚠ **Two RDS traps.** An RDS instance that you **stop** is started again automatically after 7 days, so stopping is not a way to park it. And **deleting** a database with the console's default settings creates a final snapshot that keeps costing money. This lab deletes with `--skip-final-snapshot` and `--delete-automated-backups` on purpose.

5.1. What you send to a free AI service

> ⚠ **Read this before you create a Gemini key.** Free-tier AI services are usually allowed to use your prompts to improve their products, and a human may review them. The paid tier is different. This is a normal trade for a free service, and it is also why you decide what goes into the prompt.

> Our advisor sends only a **zip code and the public forecast text** (temperature, wind, short forecast). It never sends a name, an email address, a street address or an account ID. Keep it that way. **Do not paste anything private into the advisor page while you test.** Create your key from a personal Google account, not an account that holds school or work data.

---

6. Part 0: Set Up Your `lab06` Folder

You never edit an earlier lab. You copy it to a new folder and work in the copy, so your earlier labs stay intact as a record of what you built.

0.1. Delete the previous stack

The new stack reuses the same ports as lab04, so tear down the old one first. Go into the folder of the **last lab you built** (`lab04` for undergraduates, `lab05` for graduate students) and run its delete script. Skip this if you already deleted that stack.

```
cd lab04          # graduate students: cd lab05
./delete_app_stack.sh      # graduate students: python3 delete_stack.py
cd ..
```

0.2. Copy the previous lab

Undergraduate students:

```
cp -r lab04 lab06
cd lab06
```

Graduate students:

```
cp -r lab05 lab06
cd lab06
```

0.3. Move the old application out of the way

The uploader and viewer apps and their scripts belong to the old stack. Keep them for reference, but out of the way of the new scripts.

Undergraduate students:

```
mkdir previous
mv uploader-app viewer-app create_app_stack.sh delete_app_stack.sh previous/
```

Graduate students (the Python package stays, because you will build on it):

```
mkdir previous
mv uploader-app viewer-app create_app_stack.sh delete_app_stack.sh previous/ 2>/dev/null
mv create_stack.py delete_stack.py previous/
```

Remove the old stack's settings from `.env`, because the new stack has its own:

```
grep -vE '^(UPLOADER_|VIEWER_|ALB_NAME|ALB_SG_NAME|APP_PORT|RECOVERY_SLO_SECONDS|BUCKET_NAME)' .env > .env.new
mv .env.new .env
```

Your `.env` should still hold `AWS_REGION`, `AZ_A`, `AZ_B`, `AMI_ID`, `INSTANCE_TYPE`, `KEY_NAME` and `APP_SECURITY_GROUP_ID`. Confirm that before you go on:

```
grep -E '^(AWS_REGION|AZ_A|AZ_B|AMI_ID|INSTANCE_TYPE|KEY_NAME|APP_SECURITY_GROUP_ID)=' .env
```

0.4. Your `lab06` layout

```
lab06/
  .env                          (updated in Part B)
  .env.example
  .gitignore
  <your-key>.pem
  weather-app/                  (new: app.js, package.json)
  advisor-app/                  (new: app.js, package.json)
  create_database.sh            (new)
  delete_database.sh            (new)
  create_weather_stack.sh       (new)
  delete_weather_stack.sh       (new)
  testing/
    app_smoke_test.py           (new)
  previous/                     (the old apps and scripts, for reference)
```

Graduate students also keep their `stack/` package, `tests/`, `pytest.ini` and `requirements.txt` from lab05.

✅ **Checkpoint:** `ls` inside `lab06` shows the layout above, and `previous/` holds the old uploader and viewer.

---

7. Part A: Meet the Two Weather APIs

Before you write any code, call both APIs by hand so you know what the app has to deal with. These calls go from your own computer and use no AWS resources.

A1. Zip code to coordinates

The National Weather Service API takes a **latitude and longitude**, not a zip code. So the weather service first asks a free zip lookup service for the coordinates.

```
curl -s https://api.zippopotam.us/us/60616 | python3 -m json.tool
```

Find the city, the state, and the `latitude` and `longitude` of the first place. Now try a zip that does not exist:

```
curl -s -o /dev/null -w "%{http_code}\n" https://api.zippopotam.us/us/00000
```

You should see `404`. Your app will need to turn that into a clean "zip code not found" answer for its callers.

A2. Coordinates to forecast

The NWS API needs no key, but it **requires a `User-Agent` header that identifies you**, and it refuses requests without one. Use your own email address.

```
UA="(itmo-444-544 weather lab, you@example.com)"
curl -s -H "User-Agent: $UA" "https://api.weather.gov/points/41.8458,-87.6272" | python3 -m json.tool | head -40
```

The answer does not contain the forecast. It contains **links** to it. Find `properties.forecast` and call that URL:

```
FORECAST_URL=$(curl -s -H "User-Agent: $UA" "https://api.weather.gov/points/41.8458,-87.6272" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['properties']['forecast'])")
curl -s -H "User-Agent: $UA" "$FORECAST_URL" | python3 -m json.tool | head -40
```

`properties.periods` is a list of forecast periods. The first one is "now" or "tonight". The app uses its `temperature`, `temperatureUnit`, `shortForecast`, `windSpeed` and `probabilityOfPrecipitation`.

A3. What this teaches about the weather service

Fact	What the code must do
Zip codes are not accepted by NWS	Look up coordinates first (two upstream calls per request)
A `User-Agent` is required	Send one from an environment variable, with a contact address
Either upstream can be slow or fail	Use a timeout and one retry, and report which stage failed
NWS covers only the United States and its territories	Return a clear "no coverage" answer, not a crash
The forecast can change at any time	Do not cache in the app, keep it simple

✅ **Checkpoint:** You can name the two upstream calls the weather service makes for one zip code, and say why one call is not enough.

---

8. Part B: Add the New Settings to `.env`

Append the new keys. Port 8080 serves the weather service and port 8081 serves the advisor page, matching the ports you used in lab04.

```
cat >> .env << 'EOF'

# Weather advisor lab
WEATHER_ROLE_NAME=itmo-444-544-weather-role
WEATHER_PROFILE_NAME=itmo-444-544-weather-profile
WEATHER_LT_NAME=itmo-444-544-weather-lt
WEATHER_TG_NAME=itmo-444-544-weather-tg
```

```
WEATHER_ASG_NAME=itmo-444-544-weather-asg
WEATHER_LISTENER_PORT=8080
ADVISOR_ROLE_NAME=itmo-444-544-advisor-role
ADVISOR_PROFILE_NAME=itmo-444-544-advisor-profile
ADVISOR_LT_NAME=itmo-444-544-advisor-lt
ADVISOR_TG_NAME=itmo-444-544-advisor-tg
ADVISOR_ASG_NAME=itmo-444-544-advisor-asg
ADVISOR_LISTENER_PORT=8081
ALB_NAME=itmo-444-544-weather-alb
ALB_SG_NAME=itmo-444-544-weather-alb-sg
DB_INSTANCE_ID=itmo-444-544-weather-db
DB_NAME=weather
DB_USER=weatherapp
DB_SG_NAME=itmo-444-544-weather-db-sg
DB_SUBNET_GROUP_NAME=itmo-444-544-weather-db-subnets
DB_PASSWORD_PARAM=/itmo-444-544/db-password
CACHE_SECONDS=600
GEMINI_PARAM_NAME=/itmo-444-544/gemini-api-key
GEMINI_MODEL=gemini-3.1-flash-lite
KEY_CACHE_SECONDS=60
CONTACT_EMAIL=replace-with-your-email@example.com
EOF
```

Edit `CONTACT_EMAIL` to an address you own. It goes into the NWS `User-Agent`, so the weather service can be contacted if it misbehaves.

`GEMINI_MODEL` is a setting on purpose. Model names change over time and free-tier access differs by model. If the advisor reports that the model was not found, open the model list in Google AI Studio and put a current **Flash-Lite** or **Flash** model name here. `KEY_CACHE_SECONDS=60` is short so the key experiment in Part H takes minutes instead of hours.

`DB_USER` is both the name of the database login and the master user created with the instance. `CACHE_SECONDS=600` means a saved forecast is reused for 10 minutes. You never type the database password anywhere: the create script generates it at random and stores it in SSM.

---

9. Part C: Get a Gemini Key and Store It Safely

C1. Create a free key

1. Sign in to Google AI Studio with a **personal** Google account (see the data warning in Section 5.1).
2. Create an API key from the API keys page. Copy it. It starts with `AIza`.
3. Treat it like a password. Never paste it into a file in your lab folder, a chat, or a screenshot.

C2. Test it once from your computer

Read the key into a shell variable without echoing it, and make one request. Because you type the key at a hidden prompt, it does not land in your shell history.

```
export $(grep -v '^#' .env | xargs)
read -rs -p "Gemini API key: " GEMINI_API_KEY; echo
export GEMINI_API_KEY

curl -s "https://generativelanguage.googleapis.com/v1beta/models/${GEMINI_MODEL}:generateContent" \
  -H "x-goog-api-key: $GEMINI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"contents":[{"parts":[{"text":"Suggest what to wear in 45 F weather with light rain, in one sentence."}]}]}' \
  | python3 -m json.tool | head -30
```

The text of the answer is in `candidates[0].content.parts[0].text`. If you see an error instead, the message names the problem. A `404` usually means the model name needs updating, a `400` or `403` means the key was not accepted, and a `429` means you are over the free-tier rate limit.

C3. Store it in SSM Parameter Store

A **SecureString** parameter is encrypted at rest. Reading it requires `ssm:GetParameter`, which you will grant to exactly one role, for exactly one parameter.

```
export AWS_DEFAULT_REGION="$AWS_REGION"
aws ssm put-parameter --name "$GEMINI_PARAM_NAME" --type SecureString \
  --value "$GEMINI_API_KEY" --overwrite
unset GEMINI_API_KEY
```

Check that it is stored and encrypted, without printing it:

```
aws ssm describe-parameters \
  --parameter-filters "Key=Name,Values=$GEMINI_PARAM_NAME" \
  --query "Parameters[].{Name:Name,Type:Type,Tier:Tier}" --output table
```

> 💡 **Why not put the key in `.env` or the user-data?** User-data is readable by anything on the instance and by anyone who can describe the launch template. A `.env` file gets committed to Git by accident. A SecureString parameter is read at run time by the instance role and never appears in your files.

✅ **Checkpoint:** `describe-parameters` shows `SecureString`, and your shell no longer has `GEMINI_API_KEY` set (`echo "${GEMINI_API_KEY:-unset}"` prints `unset`).

---

10. Part D: The Two Applications

Both are small Node.js programs. They write **one JSON object per log line** to standard output, which systemd keeps in the instance's journal. Read the walkthrough first, then create the files. The database itself is created in Part E, but the code that uses it is part of the weather service, so you write it here.

D1. How the weather service works

- `GET /` returns a small JSON description. The load balancer's health check uses this route.
- `GET /weather?zip=60616` validates the zip (exactly five digits), looks up coordinates, asks NWS for the grid point, then asks for the forecast and returns the first period.
- Each upstream call has an **8 second timeout** and **one retry**. A failure is reported with the stage that failed, such as `zip_lookup` or `forecast`, so the caller and the logs say *where* it broke.
- Status codes are deliberate: `400` for a bad zip, `404` for an unknown zip or a place NWS does not cover, and `502` when an upstream service failed.
- **The database is a cache and a history table.** Before calling any upstream service, the weather service asks PostgreSQL whether this zip was looked up in the last `CACHE_SECONDS`. If so it answers from the database (`"cache": "hit"`) and makes no upstream calls at all. Otherwise it fetches the forecast, saves it, and answers (`"cache": "miss"`). `GET /history` shows the ten most recent saved lookups.
- **The database is optional.** If it is down, unreachable or not configured, the service logs a `db_error`, answers without it (`"cache": "unavailable"`), and **leaves the database alone for 30 seconds** so every request does not wait for a timeout. A database problem must never become a weather outage.
- **Every query uses placeholders** (`$1`, `$2`) and never builds SQL by pasting in user input. That is what prevents SQL injection.
- **The connection is encrypted and verified.** The app connects with TLS and checks the server's certificate against Amazon's RDS certificate bundle, which the instance downloads at boot. Encryption alone would stop eavesdropping, but only the certificate check shows you are talking to your database and not to something in the middle. Statements time out after 3 seconds.
- The app creates its table when it first connects (`CREATE TABLE IF NOT EXISTS`), so there is no separate schema step.

Create `weather-app/package.json`:

```
{
  "name": "weather-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "@aws-sdk/client-ssm": "^3.600.0",
    "express": "^4.19.2",
    "pg": "^8.12.0"
  }
}
```

Create `weather-app/app.js`:

```
const fs = require('fs');
const express = require('express');
const { Pool } = require('pg');
const { SSMClient, GetParameterCommand } = require('@aws-sdk/client-ssm');

const app = express();
const PORT = process.env.PORT || 3000;
const ZIP_API = process.env.ZIP_API_BASE || 'https://api.zippopotam.us';
const NWS_API = process.env.NWS_API_BASE || 'https://api.weather.gov';
// The National Weather Service asks every client to identify itself.
const USER_AGENT = process.env.NWS_USER_AGENT || '(itmo-444-544 weather lab, student@example.com)';

// Database settings. With no DB_HOST the service still works, it just does not store anything.
const DB_HOST = process.env.DB_HOST;
const DB_PORT = Number(process.env.DB_PORT || 5432);
const DB_NAME = process.env.DB_NAME || 'weather';
const DB_USER = process.env.DB_USER || 'weatherapp';
const DB_PASSWORD_PARAM = process.env.DB_PASSWORD_PARAM || '/itmo-444-544/db-password';
const DB_CA_FILE = process.env.DB_CA_FILE || '/opt/app/rds-ca.pem';
const CACHE_SECONDS = Number(process.env.CACHE_SECONDS || 600);
const DB_RETRY_MS = 30000; // after a database failure, leave the database alone for 30 seconds

const ssm = new SSMClient({ region: process.env.AWS_REGION });

const log = (event, fields = {}) =>
  console.log(JSON.stringify({ ts: new Date().toISOString(), event, ...fields }));

class UpstreamError extends Error {
  constructor(stage, status) {
    super(`${stage} failed`);
    this.stage = stage;
    this.status = status;
  }
}

// Fetch JSON with a timeout and one retry. Returns null on a 404, throws UpstreamError otherwise.
async function fetchJson(stage, url, headers = {}) {
  let status = 0;
  for (let attempt = 1; attempt <= 2; attempt++) {
    try {
      const res = await fetch(url, { headers, signal: AbortSignal.timeout(8000) });
      status = res.status;
      if (res.ok) return await res.json();
      if (res.status === 404) return null;
      if (res.status < 500 && res.status !== 429) break; // a client error will not fix itself
    } catch (err) {
      status = 0; // timeout or network failure
```

```
    }
    if (attempt < 2) await new Promise((resolve) => setTimeout(resolve, 500));
  }
  throw new UpstreamError(stage, status);
}

// ---- Database helpers ------------------------------------------------------------

let poolPromise = null;
let dbRetryAt = 0;

async function readDbPassword() {
  if (process.env.DB_PASSWORD) return process.env.DB_PASSWORD; // local development only
  const out = await ssm.send(new GetParameterCommand({ Name: DB_PASSWORD_PARAM, WithDecryption: true }));
  return out.Parameter.Value;
}

async function createPool() {
  const password = await readDbPassword();
  // Verify the server certificate against the RDS CA bundle, so the connection is encrypted
  // and we know we are talking to RDS and not to something in the middle.
  const ssl = process.env.DB_SSL === 'off' ? false : { ca: fs.readFileSync(DB_CA_FILE, 'utf8') };
  const pool = new Pool({
    host: DB_HOST, port: DB_PORT, database: DB_NAME, user: DB_USER, password, ssl,
    max: 4, connectionTimeoutMillis: 3000, idleTimeoutMillis: 30000, statement_timeout: 3000,
  });
  pool.on('error', (err) => log('db_error', { stage: 'pool', message: err.message }));
  try {
    await pool.query(`CREATE TABLE IF NOT EXISTS lookups (
      id BIGSERIAL PRIMARY KEY,
      zip CHAR(5) NOT NULL,
      payload JSONB NOT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now())`);
    await pool.query('CREATE INDEX IF NOT EXISTS lookups_zip_created ON lookups (zip, created_at DESC)');
  } catch (err) {
    await pool.end().catch(() => {});
    throw err;
  }
  return pool;
}

// Returns a ready pool, or throws. A failed attempt is not remembered, so the next try starts fresh.
function getPool() {
  if (!poolPromise) {
    poolPromise = createPool().catch((err) => {
      poolPromise = null;
      throw err;
    });
  }
  return poolPromise;
}

// Run one database call. Any failure is logged, and the database is skipped for a short time,
// so a broken database costs each request nothing instead of a 3 second wait.
async function withDb(stage, fn) {
  if (!DB_HOST) return { state: 'disabled' };
  if (Date.now() < dbRetryAt) return { state: 'unavailable' };
  try {
    return { state: 'ok', value: await fn(await getPool()) };
  } catch (err) {
    dbRetryAt = Date.now() + DB_RETRY_MS;
    log('db_error', { stage, message: err.message });
    return { state: 'unavailable' };
  }
}

const findFresh = (zip) => withDb('read', async (pool) => {
  const r = await pool.query(
    `SELECT payload FROM lookups
      WHERE zip = $1 AND created_at > now() - make_interval(secs => $2)
      ORDER BY created_at DESC LIMIT 1`, [zip, CACHE_SECONDS]);
  return r.rows.length ? r.rows[0].payload : null;
});

const saveLookup = (zip, payload) => withDb('write', (pool) =>
  pool.query('INSERT INTO lookups (zip, payload) VALUES ($1, $2)', [zip, JSON.stringify(payload)]));

// ---- Routes ----------------------------------------------------------------------

app.get('/', (req, res) => {
  res.json({ service: 'weather', usage: '/weather?zip=60616', history: '/history' });
});

// The ten most recent lookups that were saved in the database.
app.get('/history', async (req, res) => {
  const result = await withDb('history', async (pool) => {
    const r = await pool.query(
      `SELECT id, zip, payload->>'city' AS city, payload->>'state' AS state,
              payload->'forecast'->>'shortForecast' AS short_forecast,
              payload->'forecast'->>'temperature' AS temperature, created_at
         FROM lookups ORDER BY created_at DESC LIMIT 10`);
```

```
    return r.rows;
  });
  if (result.state === 'disabled') return res.status(503).json({ error: 'database_disabled' });
  if (result.state === 'unavailable') return res.status(503).json({ error: 'database_unavailable' });
  return res.json({ count: result.value.length, lookups: result.value });
});

app.get('/weather', async (req, res) => {
  const zip = String(req.query.zip || '');
  if (!/^\d{5}$/.test(zip)) {
    log('weather_rejected', { reason: 'invalid_zip' });
    return res.status(400).json({ error: 'invalid_zip', message: 'Give a 5 digit US zip code.' });
  }

  const started = Date.now();
  log('weather_request', { zip });

  // 0. Has anyone asked for this zip in the last few minutes? Then answer from the database.
  const cached = await findFresh(zip);
  if (cached.state === 'ok' && cached.value) {
    log('weather_success', { zip, cache: 'hit', ms: Date.now() - started });
    return res.json({ ...cached.value, cache: 'hit' });
  }

  try {
    // 1. The weather API wants coordinates, not zip codes, so look the zip code up first.
    const lookup = await fetchJson('zip_lookup', `${ZIP_API}/us/${zip}`);
    if (!lookup || !lookup.places || lookup.places.length === 0) {
      log('weather_rejected', { reason: 'zip_not_found', zip });
      return res.status(404).json({ error: 'zip_not_found' });
    }
    const place = lookup.places[0];
    const lat = Number(place.latitude).toFixed(4);
    const lon = Number(place.longitude).toFixed(4);

    // 2. Ask NWS which forecast grid covers those coordinates.
    const headers = { 'User-Agent': USER_AGENT, Accept: 'application/geo+json' };
    const point = await fetchJson('nws_points', `${NWS_API}/points/${lat},${lon}`, headers);
    if (!point) {
      log('weather_rejected', { reason: 'no_nws_coverage', zip });
      return res.status(404).json({ error: 'no_nws_coverage' });
    }

    // 3. Fetch the forecast for that grid and keep the first period (now, or the next block).
    const forecast = await fetchJson('nws_forecast', point.properties.forecast, headers);
    const period = forecast.properties.periods[0];
    const precip = period.probabilityOfPrecipitation ? period.probabilityOfPrecipitation.value : null;

    const body = {
      zip,
      city: place['place name'],
      state: place['state abbreviation'],
      latitude: lat,
      longitude: lon,
      forecast: {
        name: period.name,
        temperature: period.temperature,
        temperatureUnit: period.temperatureUnit,
        shortForecast: period.shortForecast,
        windSpeed: period.windSpeed,
        precipitationChance: precip,
        isDaytime: period.isDaytime,
      },
    };

    // 4. Keep a copy. This is both the cache for the next caller and the history table.
    const saved = await saveLookup(zip, body);
    const cache = saved.state === 'ok' ? 'miss' : saved.state; // 'miss', 'disabled' or 'unavailable'
    log('weather_success', { zip, cache, ms: Date.now() - started });
    return res.json({ ...body, cache });
  } catch (err) {
    if (err instanceof UpstreamError) {
      log('weather_error', { zip, stage: err.stage, status: err.status });
      return res.status(502).json({ error: 'upstream_error', stage: err.stage });
    }
    log('weather_error', { zip, stage: 'internal', message: err.message });
    return res.status(500).json({ error: 'internal_error' });
  }
});

app.listen(PORT, () => log('weather_started', { port: PORT }));
```

D2. How the advisor service works

- `GET /advice?zip=60616` returns JSON. `GET /` serves a small web page with a form, and shows the result when a zip is given.
- It calls the weather service at `WEATHER_URL` (the ALB, port 8080). Bad zips and unknown zips pass through as `400` and `404`. Any other weather failure becomes `502 weather_service_unavailable`.
- It reads the Gemini key from SSM with the instance role, and **caches it for `KEY_CACHE_SECONDS`**, so it does not call SSM on every request.
- The prompt contains only the zip code, city, state and forecast fields. See Section 5.1.

- If Gemini fails for any reason (bad key, rate limit, timeout, empty answer), the advisor **logs a `gemini_error` and answers with a rule-based suggestion**. The response says `"source": "fallback"` so nobody mistakes it for AI output.
- The home page always returns `200`, even if the weather service is down, so a weather outage never makes the load balancer replace a healthy advisor instance.
- Everything printed on the page goes through `escapeHtml`. The AI's text is **untrusted input**. A model can be tricked into returning markup, and a page that prints it unescaped is a cross-site scripting hole.

Create `advisor-app/package.json`:

```
{
  "name": "advisor-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.19.2",
    "@aws-sdk/client-ssm": "^3.600.0"
  }
}
```

Create `advisor-app/app.js`:

```
const express = require('express');
const { SSMClient, GetParameterCommand } = require('@aws-sdk/client-ssm');

const app = express();
const PORT = process.env.PORT || 3000;
const WEATHER_URL = process.env.WEATHER_URL; // for example http://<alb-dns>:8080
const GEMINI_API_BASE = process.env.GEMINI_API_BASE || 'https://generativelanguage.googleapis.com';
const GEMINI_MODEL = process.env.GEMINI_MODEL || 'gemini-3.1-flash-lite';
const KEY_PARAM = process.env.GEMINI_KEY_PARAM || '/itmo-444-544/gemini-api-key';
const KEY_CACHE_MS = Number(process.env.KEY_CACHE_SECONDS || 300) * 1000;

const ssm = new SSMClient({ region: process.env.AWS_REGION });
let cachedKey = { value: null, at: 0 };

const log = (event, fields = {}) =>
  console.log(JSON.stringify({ ts: new Date().toISOString(), event, ...fields }));

// The API key lives in SSM Parameter Store, never in user-data or an environment file.
// GEMINI_API_KEY is for running this app on your own laptop only.
async function getApiKey() {
  if (process.env.GEMINI_API_KEY) return process.env.GEMINI_API_KEY;
  if (cachedKey.value && Date.now() - cachedKey.at < KEY_CACHE_MS) return cachedKey.value;
  const out = await ssm.send(new GetParameterCommand({ Name: KEY_PARAM, WithDecryption: true }));
  cachedKey = { value: out.Parameter.Value, at: Date.now() };
  return cachedKey.value;
}

const SYSTEM_PROMPT =
  'You give a short clothing suggestion for the weather you are given. ' +
  'Reply in at most two plain sentences, with no markdown and no lists. Use only the data provided.';

// Only the zip code and the forecast go to the AI service. Nothing about the user does.
function buildPrompt(w) {
  const f = w.forecast;
  return [
    `Zip code: ${w.zip} (${w.city}, ${w.state})`,
    `Period: ${f.name}`,
    `Temperature: ${f.temperature} ${f.temperatureUnit}`,
    `Conditions: ${f.shortForecast}`,
    `Wind: ${f.windSpeed}`,
    `Chance of precipitation: ${f.precipitationChance === null ? 'unknown' : f.precipitationChance + ' percent'}`,
  ].join('\n');
}

async function askGemini(prompt) {
  const key = await getApiKey();
  const res = await fetch(`${GEMINI_API_BASE}/v1beta/models/${GEMINI_MODEL}:generateContent`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'x-goog-api-key': key },
    body: JSON.stringify({
      systemInstruction: { parts: [{ text: SYSTEM_PROMPT }] },
      contents: [{ parts: [{ text: prompt }] }],
      generationConfig: { maxOutputTokens: 300 },
    }),
    signal: AbortSignal.timeout(15000),
  });
  if (!res.ok) {
    const err = new Error('gemini_http_error');
    err.status = res.status;
    throw err;
  }
  const data = await res.json();
  const parts = (data.candidates && data.candidates[0] && data.candidates[0].content.parts) || [];
  const text = parts.map((p) => p.text || '').join('').trim();
  if (!text) {
    const err = new Error('gemini_empty_response');
    err.status = 200;
    throw err;
  }
```

```
  return text;
}

// Used when the AI service is unavailable, rate limited, or the key is wrong.
function fallbackAdvice(f) {
  const t = f.temperature;
  let base;
  if (t <= 32) base = 'Wear a heavy coat, a hat and gloves.';
  else if (t <= 50) base = 'Wear a warm jacket and long pants.';
  else if (t <= 68) base = 'Wear a light jacket or a sweater over long sleeves.';
  else if (t <= 80) base = 'A t-shirt and light pants or shorts will be fine.';
  else base = 'Wear light, breathable clothes and bring water and sunscreen.';
  const wet = (f.precipitationChance || 0) >= 40 || /rain|shower|storm|snow/i.test(f.shortForecast);
  return wet ? `${base} Bring an umbrella or a waterproof jacket.` : base;
}

const escapeHtml = (s) =>
  String(s).replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

// Shared by the web page and the JSON endpoint. Returns { status, body }.
async function getAdvice(zip) {
  if (!/^\d{5}$/.test(zip)) {
    log('advice_rejected', { reason: 'invalid_zip' });
    return { status: 400, body: { error: 'invalid_zip', message: 'Give a 5 digit US zip code.' } };
  }
  const started = Date.now();
  log('advice_request', { zip });

  let weather;
  try {
    const res = await fetch(`${WEATHER_URL}/weather?zip=${zip}`, { signal: AbortSignal.timeout(20000) });
    if (res.status === 400 || res.status === 404) {
      log('advice_rejected', { reason: 'weather_' + res.status, zip });
      return { status: res.status, body: await res.json() };
    }
    if (!res.ok) throw new Error('weather_status_' + res.status);
    weather = await res.json();
  } catch (err) {
    log('weather_call_failed', { zip, message: err.message });
    return { status: 502, body: { error: 'weather_service_unavailable' } };
  }

  let advice, source;
  try {
    advice = await askGemini(buildPrompt(weather));
    source = 'gemini';
  } catch (err) {
    log('gemini_error', { zip, status: err.status || 0, reason: err.name === 'TimeoutError' ? 'timeout' : err.message });
    advice = fallbackAdvice(weather.forecast);
    source = 'fallback';
  }

  log('advice_success', { zip, source, ms: Date.now() - started });
  return { status: 200, body: { ...weather, advice, source, model: source === 'gemini' ? GEMINI_MODEL : null } };
}

app.get('/advice', async (req, res) => {
  const { status, body } = await getAdvice(String(req.query.zip || ''));
  res.status(status).json(body);
});

app.get('/', async (req, res) => {
  const form = `<form method="GET" action="/">
      <label>US zip code <input name="zip" pattern="[0-9]{5}" maxlength="5" required></label>
      <button type="submit">What should I wear?</button>
    </form>`;
  let result = '';
  if (req.query.zip) {
    const { status, body } = await getAdvice(String(req.query.zip));
    if (status === 200) {
      const f = body.forecast;
      result = `<h2>${escapeHtml(body.city)}, ${escapeHtml(body.state)} ${escapeHtml(body.zip)}</h2>
        <p>${escapeHtml(f.name)}: ${escapeHtml(f.temperature)} ${escapeHtml(f.temperatureUnit)}, ${escapeHtml(f.shortForecast)}, wind
${escapeHtml(f.windSpeed)}</p>
        <p><strong>${escapeHtml(body.advice)}</strong></p>
        <p><small>Suggestion source: ${escapeHtml(body.source)}</small></p>`;
    } else {
      result = `<p>Sorry: ${escapeHtml(body.error)}</p>`;
    }
  }
  res.send(`<h1>Weather Advisor</h1>${form}${result}`);
});

app.listen(PORT, () => log('advisor_started', { port: PORT, model: GEMINI_MODEL }));
```

D3. Optional: run both apps on your own computer first

If you have Node.js 18 or newer, you can try them locally before touching AWS. Use your key through a variable for local testing only.

```
(cd weather-app && npm install --omit=dev && PORT=3001 node app.js) &
```

```
(cd advisor-app && npm install --omit=dev && PORT=3002 WEATHER_URL=http://localhost:3001 \
  GEMINI_API_KEY="$YOUR_KEY" node app.js) &
curl -s "http://localhost:3002/advice?zip=60616" | python3 -m json.tool
```

Stop them with `kill %1 %2` when you are done.

---

11. Part E: The Database

Graduate students: these scripts are your specification. Write the Python version (Section 3).

E1. What you are creating, and why each setting is there

Setting	Value	Why
Engine	`postgres`	Your choice for this lab. The default current major version is fine
Instance class	`db.t4g.micro`	The free-tier size. A bigger class is billed
Storage	20 GB, `gp2`, **encrypted**	20 GB is the free-tier amount. Encryption with the default AWS managed key is free
Availability	`--no-multi-az`, one zone (`AZ_A`)	A second copy in another zone doubles the cost and is **not** free. This is a lab
Reachability	`--no-publicly-accessible`, in the default VPC subnets	The database has no public address. Only things inside the VPC can even try to connect
Network rule	Security group allowing port **5432 only from the app security group**	The same "name a security group as the source" idea as the ALB rule
Backups	**`--backup-retention-period 0`**	**Automated backups and point-in-time recovery are off, so no backup storage is used and no snapshot images are created.**
Deletion protection	off	So the cleanup script can delete it. Production databases keep this on
Password	random, stored as an SSM SecureString, never printed or saved to a file	See below

What retention 0 costs you. With no backups there is no way to restore this database if it is damaged or deleted. That is acceptable here because the data is a cache of public forecasts plus a history table that you generate while testing, and because the whole database is deleted at the end. For a database that holds data you care about, you would **never** do this. You would keep several days of automated backups and practice restoring them.

The password. `openssl rand -hex 16` makes 32 random characters. The script writes them straight into SSM Parameter Store and uses them once in the create call. A value passed on a command line can be seen for a moment by other users of the same machine (in a process list). On your own laptop that is fine. On a shared server you would use RDS's managed password (`--manage-master-user-password`) instead, which keeps it in AWS Secrets Manager. Secrets Manager is not free, so we accept the small exposure.

Why separate scripts. The database takes 5 to 10 minutes to appear and is deleted at a different time from the stack, so it has its own pair of scripts. The create script writes the database address to `db_outputs.env`, which the stack script reads.

E2. `create_database.sh`

Create `create_database.sh`:

```
#!/bin/bash
set -euo pipefail

export $(grep -v '^#' .env | xargs)
export AWS_DEFAULT_REGION="$AWS_REGION"   # lets us skip --region on every call

echo "== 1. Default VPC and subnets in both AZs =="
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text)
SUBNET_A=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_A" \
  --query "Subnets[0].SubnetId" --output text)
SUBNET_B=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_B" \
  --query "Subnets[0].SubnetId" --output text)

echo "== 2. Database security group (port 5432, from the app security group only) =="
DB_SECURITY_GROUP_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$DB_SG_NAME" "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text 2>/dev/null || true)
if [ -z "$DB_SECURITY_GROUP_ID" ] || [ "$DB_SECURITY_GROUP_ID" = "None" ]; then
  DB_SECURITY_GROUP_ID=$(aws ec2 create-security-group \
    --group-name "$DB_SG_NAME" \
    --description "PostgreSQL access from the lab app instances only" \
    --vpc-id "$VPC_ID" --query "GroupId" --output text)
  aws ec2 authorize-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
    --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID" >/dev/null
else
  echo "Database security group already exists, skipping."
fi

echo "== 3. DB subnet group =="
aws rds create-db-subnet-group \
  --db-subnet-group-name "$DB_SUBNET_GROUP_NAME" \
```

```
  --db-subnet-group-description "Lab subnets for the weather database" \
  --subnet-ids "$SUBNET_A" "$SUBNET_B" >/dev/null 2>&1 || echo "DB subnet group already exists, skipping."

echo "== 4. Database password (random, stored as a SecureString, never printed) =="
if ! aws ssm get-parameter --name "$DB_PASSWORD_PARAM" >/dev/null 2>&1; then
  aws ssm put-parameter --name "$DB_PASSWORD_PARAM" --type SecureString \
    --value "$(openssl rand -hex 16)" >/dev/null
else
  echo "Password parameter already exists, keeping it."
fi
DB_PASSWORD=$(aws ssm get-parameter --name "$DB_PASSWORD_PARAM" --with-decryption \
  --query Parameter.Value --output text)

echo "== 5. The PostgreSQL instance (free tier size, no backups) =="
if aws rds describe-db-instances --db-instance-identifier "$DB_INSTANCE_ID" >/dev/null 2>&1; then
  echo "Database $DB_INSTANCE_ID already exists, skipping."
else
  aws rds create-db-instance \
    --db-instance-identifier "$DB_INSTANCE_ID" \
    --db-instance-class db.t4g.micro \
    --engine postgres \
    --allocated-storage 20 --storage-type gp2 \
    --master-username "$DB_USER" --master-user-password "$DB_PASSWORD" \
    --db-name "$DB_NAME" \
    --db-subnet-group-name "$DB_SUBNET_GROUP_NAME" \
    --vpc-security-group-ids "$DB_SECURITY_GROUP_ID" \
    --availability-zone "$AZ_A" \
    --backup-retention-period 0 \
    --no-multi-az \
    --no-publicly-accessible \
    --no-auto-minor-version-upgrade \
    --no-deletion-protection \
    --storage-encrypted >/dev/null
fi
unset DB_PASSWORD

echo "Waiting for the database to become available (usually 5 to 10 minutes)..."
aws rds wait db-instance-available --db-instance-identifier "$DB_INSTANCE_ID"

DB_HOST=$(aws rds describe-db-instances --db-instance-identifier "$DB_INSTANCE_ID" \
  --query "DBInstances[0].Endpoint.Address" --output text)

cat > db_outputs.env << EOF
DB_HOST=$DB_HOST
DB_SECURITY_GROUP_ID=$DB_SECURITY_GROUP_ID
EOF

echo
echo "Done. Database endpoint: $DB_HOST"
echo "Automated backups are OFF (retention 0) and no final snapshot will be taken on delete."
echo "Next: ./create_weather_stack.sh. When testing is finished: ./delete_weather_stack.sh then ./delete_database.sh"
```

E3. `delete_database.sh`

Create `delete_database.sh`:

```
#!/bin/bash
set -uo pipefail

export $(grep -v '^#' .env | xargs)
export AWS_DEFAULT_REGION="$AWS_REGION"

echo "== Deleting the database (no final snapshot, no kept backups) =="
if aws rds describe-db-instances --db-instance-identifier "$DB_INSTANCE_ID" >/dev/null 2>&1; then
  aws rds delete-db-instance --db-instance-identifier "$DB_INSTANCE_ID" \
    --skip-final-snapshot --delete-automated-backups >/dev/null
  echo "Waiting for the database to be deleted (a few minutes)..."
  aws rds wait db-instance-deleted --db-instance-identifier "$DB_INSTANCE_ID"
else
  echo "Database $DB_INSTANCE_ID not found, nothing to delete."
fi

echo "== Deleting the DB subnet group =="
aws rds delete-db-subnet-group --db-subnet-group-name "$DB_SUBNET_GROUP_NAME" 2>/dev/null || true

echo "== Deleting the database security group =="
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text)
DB_SG=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$DB_SG_NAME" "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text 2>/dev/null || true)
if [ -n "$DB_SG" ] && [ "$DB_SG" != "None" ]; then
  for i in $(seq 1 10); do
    if aws ec2 delete-security-group --group-id "$DB_SG" 2>/dev/null; then
      echo "Database security group deleted."
      break
    fi
    echo "Still releasing, waiting 15s..."
    sleep 15
  done
```

```
fi

echo "== Deleting the database password parameter =="
aws ssm delete-parameter --name "$DB_PASSWORD_PARAM" 2>/dev/null || true

rm -f db_outputs.env
echo "Database removed. Run the checks in the last experiment step to confirm no snapshots or backups remain."
```

The delete script does four things in order: delete the instance **with no final snapshot and with its automated backups removed**, wait until it is really gone, delete the DB subnet group and the database security group (neither can be deleted while the instance exists), and delete the password parameter.

Make all four scripts executable, then **start the database now**, because it builds for several minutes. You can read Part F while it runs.

```
chmod +x create_database.sh delete_database.sh create_weather_stack.sh delete_weather_stack.sh
./create_database.sh
```

✅ **Checkpoint:** The script finished and printed the database endpoint. You can explain what `--backup-retention-period 0` turns off, and why that is acceptable here but not for real data.

---

12. Part F: The Stack Scripts

> **Graduate students:** these scripts are your specification. Write the Python version (Section 3).

F1. What `create_weather_stack.sh` does

Read the order, because it is different from lab04.

1. Checks that the Gemini parameter exists and that `db_outputs.env` exists (so the database was created), and stops with a clear message if not.
2. Creates **two roles and instance profiles** (weather and advisor).
3. Gives **each role an inline policy for the one secret its tier needs**: the weather role may read the database password, and the advisor role may read the Gemini key. Neither can read the other's.
4. Finds the default VPC and a subnet in each of your two availability zones.
5. Creates the ALB security group (ports 8080 and 8081 open), and tightens the app security group so port 3000 accepts traffic **only from the ALB**.
6. Creates both target groups and the **ALB with its two listeners**.
7. Builds both launch templates. The advisor's user-data includes the ALB DNS name, which is why the ALB came first. The weather user-data includes the database address, installs the `psql` client, and downloads the RDS certificate bundle.
8. Creates both Auto Scaling Groups, which launch the instances.

Every step checks whether the thing already exists, so the script is safe to run twice.

Create `create_weather_stack.sh`:

```
#!/bin/bash
set -euo pipefail

export $(grep -v '^#' .env | xargs)
export AWS_DEFAULT_REGION="$AWS_REGION"   # lets us skip --region on every call

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
GEMINI_PARAM_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${GEMINI_PARAM_NAME}"
DB_PASSWORD_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${DB_PASSWORD_PARAM}"

# ------------------------------------------------------------------------------
# Helper functions (each one is used twice, once per tier)
# ------------------------------------------------------------------------------

# $1 = role name, $2 = instance profile name
ensure_role_and_profile() {
  local ROLE="$1" PROFILE="$2"
  aws iam create-role --role-name "$ROLE" \
    --assume-role-policy-document file://trust-policy.json >/dev/null 2>&1 || echo "Role $ROLE exists, skipping."
  aws iam create-instance-profile --instance-profile-name "$PROFILE" >/dev/null 2>&1 || echo "Profile $PROFILE exists, skipping."
  aws iam add-role-to-instance-profile --instance-profile-name "$PROFILE" --role-name "$ROLE" >/dev/null 2>&1 || true
}

# $1 = name. Prints the target group ARN (log lines go to stderr so they are not captured)
ensure_target_group() {
  local NAME="$1" ARN
  ARN=$(aws elbv2 describe-target-groups --names "$NAME" \
    --query "TargetGroups[0].TargetGroupArn" --output text 2>/dev/null || true)
  if [ -z "$ARN" ] || [ "$ARN" = "None" ]; then
    ARN=$(aws elbv2 create-target-group \
      --name "$NAME" --protocol HTTP --port 3000 --vpc-id "$VPC_ID" --target-type instance \
      --health-check-protocol HTTP --health-check-path / \
      --health-check-interval-seconds 10 --health-check-timeout-seconds 5 \
      --healthy-threshold-count 2 --unhealthy-threshold-count 2 \
      --query "TargetGroups[0].TargetGroupArn" --output text)
  else
    echo "Target group $NAME already exists, skipping." >&2
  fi
  echo "$ARN"
}
```

```
# $1 = port, $2 = target group ARN
ensure_listener() {
  local PORT="$1" TG_ARN="$2" EXISTING
  EXISTING=$(aws elbv2 describe-listeners --load-balancer-arn "$ALB_ARN" \
    --query "Listeners[].Port" --output text 2>/dev/null || true)
  if echo "$EXISTING" | tr '\t' '\n' | grep -qx "$PORT"; then
    echo "Listener on port $PORT already exists, skipping."
  else
    aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" \
      --protocol HTTP --port "$PORT" \
      --default-actions "Type=forward,TargetGroupArn=$TG_ARN" >/dev/null
  fi
}

# Writes user-data-<name>.sh
# $1 = name (weather or advisor), $2 = app folder, $3 = extra Environment= lines,
# $4 = extra setup commands to run before npm install (may be empty)
build_user_data() {
  local NAME="$1" APP_DIR="$2" EXTRA_ENV="$3" EXTRA_SETUP="${4:-}"
  local APP_JS_B64 PKG_JSON_B64
  APP_JS_B64=$(base64 -w0 "$APP_DIR/app.js")
  PKG_JSON_B64=$(base64 -w0 "$APP_DIR/package.json")
  cat > "user-data-${NAME}.sh" << EOF
#!/bin/bash
dnf update -y
dnf install -y nodejs
mkdir -p /opt/app
${EXTRA_SETUP}
echo "$APP_JS_B64" | base64 -d > /opt/app/app.js
echo "$PKG_JSON_B64" | base64 -d > /opt/app/package.json
cd /opt/app
npm install --omit=dev

cat > /etc/systemd/system/${NAME}.service << 'SERVICE'
[Unit]
Description=${NAME} app
After=network.target

[Service]
Environment=PORT=3000
Environment=AWS_REGION=$AWS_REGION
${EXTRA_ENV}
WorkingDirectory=/opt/app
ExecStart=/usr/bin/node /opt/app/app.js
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
SERVICE

chown -R ec2-user:ec2-user /opt/app
systemctl daemon-reload
systemctl enable ${NAME}.service
systemctl start ${NAME}.service
EOF
}

# $1 = launch template name, $2 = instance profile name, $3 = user-data file, $4 = Name tag
ensure_launch_template() {
  local LT_NAME="$1" PROFILE="$2" USERDATA_FILE="$3" TAG="$4"
  cat > "lt-data-${TAG}.json" << EOF
{
  "ImageId": "$AMI_ID",
  "InstanceType": "$INSTANCE_TYPE",
  "KeyName": "$KEY_NAME",
  "SecurityGroupIds": ["$APP_SECURITY_GROUP_ID"],
  "IamInstanceProfile": { "Name": "$PROFILE" },
  "UserData": "$(base64 -w0 "$USERDATA_FILE")",
  "TagSpecifications": [
    { "ResourceType": "instance", "Tags": [{ "Key": "Name", "Value": "$TAG" }] }
  ]
}
EOF
  if aws ec2 describe-launch-templates --launch-template-names "$LT_NAME" >/dev/null 2>&1; then
    echo "Template $LT_NAME exists, publishing a new version"
    aws ec2 create-launch-template-version --launch-template-name "$LT_NAME" \
      --source-version 1 --launch-template-data "file://lt-data-${TAG}.json" >/dev/null
  else
    aws ec2 create-launch-template --launch-template-name "$LT_NAME" \
      --launch-template-data "file://lt-data-${TAG}.json" >/dev/null
  fi
}

# $1 = ASG name, $2 = launch template name, $3 = target group ARN, $4 = Name tag
ensure_asg() {
  local ASG="$1" LT_NAME="$2" TG_ARN="$3" TAG="$4" EXISTING
  EXISTING=$(aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names "$ASG" \
    --query "AutoScalingGroups[0].AutoScalingGroupName" --output text 2>/dev/null || true)
  if [ -z "$EXISTING" ] || [ "$EXISTING" = "None" ]; then
```

```
    aws autoscaling create-auto-scaling-group \
      --auto-scaling-group-name "$ASG" \
      --launch-template "LaunchTemplateName=$LT_NAME,Version=\$Latest" \
      --min-size 1 --max-size 1 --desired-capacity 1 \
      --vpc-zone-identifier "$SUBNET_A,$SUBNET_B" \
      --target-group-arns "$TG_ARN" \
      --health-check-type ELB --health-check-grace-period 120 \
      --tags "ResourceId=$ASG,ResourceType=auto-scaling-group,Key=Name,Value=$TAG,PropagateAtLaunch=true"
  else
    echo "Auto Scaling Group $ASG already exists, skipping."
  fi
}

# ------------------------------------------------------------------------------
# Build
# ------------------------------------------------------------------------------

echo "== 1. Checking the Gemini key and the database =="
if ! aws ssm get-parameter --name "$GEMINI_PARAM_NAME" >/dev/null 2>&1; then
  echo "Parameter $GEMINI_PARAM_NAME was not found. Create it first (Part C of the lab)."
  exit 1
fi
if [ ! -f db_outputs.env ]; then
  echo "db_outputs.env was not found. Run ./create_database.sh first (Part E of the lab)."
  exit 1
fi
export $(grep -v '^#' db_outputs.env | xargs)   # gives us DB_HOST

echo "== 2. IAM roles and instance profiles =="
cat > trust-policy.json << 'EOF'
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Principal": { "Service": "ec2.amazonaws.com" }, "Action": "sts:AssumeRole" } ] }
EOF
ensure_role_and_profile "$WEATHER_ROLE_NAME" "$WEATHER_PROFILE_NAME"
ensure_role_and_profile "$ADVISOR_ROLE_NAME" "$ADVISOR_PROFILE_NAME"

# Each role may read only the one secret its tier needs, and nothing else.
cat > weather-policy.json << EOF
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": ["ssm:GetParameter"], "Resource": "${DB_PASSWORD_ARN}" } ] }
EOF
aws iam put-role-policy --role-name "$WEATHER_ROLE_NAME" \
  --policy-name "${WEATHER_ROLE_NAME}-policy" --policy-document file://weather-policy.json

cat > advisor-policy.json << EOF
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": ["ssm:GetParameter"], "Resource": "${GEMINI_PARAM_ARN}" } ] }
EOF
aws iam put-role-policy --role-name "$ADVISOR_ROLE_NAME" \
  --policy-name "${ADVISOR_ROLE_NAME}-policy" --policy-document file://advisor-policy.json
echo "Waiting 15s for IAM propagation..."
sleep 15

echo "== 3. Default VPC and subnets in both AZs =="
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text)
SUBNET_A=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_A" \
  --query "Subnets[0].SubnetId" --output text)
SUBNET_B=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_B" \
  --query "Subnets[0].SubnetId" --output text)

echo "== 4. Security groups =="
ALB_SECURITY_GROUP_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$ALB_SG_NAME" "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text 2>/dev/null || true)
if [ -z "$ALB_SECURITY_GROUP_ID" ] || [ "$ALB_SECURITY_GROUP_ID" = "None" ]; then
  ALB_SECURITY_GROUP_ID=$(aws ec2 create-security-group \
    --group-name "$ALB_SG_NAME" \
    --description "Allow inbound to the ALB on the app listener ports" \
    --vpc-id "$VPC_ID" --query "GroupId" --output text)
  aws ec2 authorize-security-group-ingress --group-id "$ALB_SECURITY_GROUP_ID" \
    --protocol tcp --port "$WEATHER_LISTENER_PORT" --cidr 0.0.0.0/0 >/dev/null
  aws ec2 authorize-security-group-ingress --group-id "$ALB_SECURITY_GROUP_ID" \
    --protocol tcp --port "$ADVISOR_LISTENER_PORT" --cidr 0.0.0.0/0 >/dev/null
else
  echo "ALB security group already exists, skipping."
fi
aws ec2 revoke-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --cidr 0.0.0.0/0 >/dev/null 2>&1 || true
aws ec2 authorize-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --source-group "$ALB_SECURITY_GROUP_ID" >/dev/null 2>&1 || true

echo "== 5. Target groups and the load balancer (the ALB comes first this time) =="
WEATHER_TG_ARN=$(ensure_target_group "$WEATHER_TG_NAME")
ADVISOR_TG_ARN=$(ensure_target_group "$ADVISOR_TG_NAME")

ALB_ARN=$(aws elbv2 describe-load-balancers --names "$ALB_NAME" \
  --query "LoadBalancers[0].LoadBalancerArn" --output text 2>/dev/null || true)
if [ -z "$ALB_ARN" ] || [ "$ALB_ARN" = "None" ]; then
```

```
  ALB_ARN=$(aws elbv2 create-load-balancer \
    --name "$ALB_NAME" --subnets "$SUBNET_A" "$SUBNET_B" \
    --security-groups "$ALB_SECURITY_GROUP_ID" \
    --scheme internet-facing --type application \
    --query "LoadBalancers[0].LoadBalancerArn" --output text)
  echo "Waiting for the ALB to become active..."
  aws elbv2 wait load-balancer-available --load-balancer-arns "$ALB_ARN"
else
  echo "Load balancer $ALB_NAME already exists, skipping."
fi
ensure_listener "$WEATHER_LISTENER_PORT" "$WEATHER_TG_ARN"
ensure_listener "$ADVISOR_LISTENER_PORT" "$ADVISOR_TG_ARN"
ALB_DNS_NAME=$(aws elbv2 describe-load-balancers --load-balancer-arns "$ALB_ARN" \
  --query "LoadBalancers[0].DNSName" --output text)

echo "== 6. Launch templates (the advisor needs the ALB address to reach the weather tier) =="
WEATHER_ENV="Environment=\"NWS_USER_AGENT=(itmo-444-544 weather lab, ${CONTACT_EMAIL})\"
Environment=DB_HOST=${DB_HOST}
Environment=DB_NAME=${DB_NAME}
Environment=DB_USER=${DB_USER}
Environment=DB_PASSWORD_PARAM=${DB_PASSWORD_PARAM}
Environment=CACHE_SECONDS=${CACHE_SECONDS}"
# The RDS certificate bundle (so the app can verify the database) and the psql client for your own queries.
WEATHER_SETUP="dnf install -y postgresql15
curl -sS -o /opt/app/rds-ca.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem"
ADVISOR_ENV="Environment=WEATHER_URL=http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}
Environment=GEMINI_KEY_PARAM=${GEMINI_PARAM_NAME}
Environment=GEMINI_MODEL=${GEMINI_MODEL}
Environment=KEY_CACHE_SECONDS=${KEY_CACHE_SECONDS}"

build_user_data weather weather-app "$WEATHER_ENV" "$WEATHER_SETUP"
build_user_data advisor advisor-app "$ADVISOR_ENV"
ensure_launch_template "$WEATHER_LT_NAME" "$WEATHER_PROFILE_NAME" user-data-weather.sh weather-app
ensure_launch_template "$ADVISOR_LT_NAME" "$ADVISOR_PROFILE_NAME" user-data-advisor.sh advisor-app

echo "== 7. Auto Scaling Groups (these launch the instances) =="
ensure_asg "$WEATHER_ASG_NAME" "$WEATHER_LT_NAME" "$WEATHER_TG_ARN" weather-app
ensure_asg "$ADVISOR_ASG_NAME" "$ADVISOR_LT_NAME" "$ADVISOR_TG_ARN" advisor-app

cat > stack_outputs.env << EOF
VPC_ID=$VPC_ID
SUBNET_A=$SUBNET_A
SUBNET_B=$SUBNET_B
ALB_SECURITY_GROUP_ID=$ALB_SECURITY_GROUP_ID
ALB_ARN=$ALB_ARN
ALB_DNS_NAME=$ALB_DNS_NAME
WEATHER_TG_ARN=$WEATHER_TG_ARN
ADVISOR_TG_ARN=$ADVISOR_TG_ARN
EOF
rm -f trust-policy.json weather-policy.json advisor-policy.json user-data-weather.sh user-data-advisor.sh lt-data-*.json

echo
echo "Done. ALB DNS name: $ALB_DNS_NAME"
echo "Weather service: http://$ALB_DNS_NAME:$WEATHER_LISTENER_PORT/weather?zip=60616   (saved lookups: /history)"
echo "Advisor page:    http://$ALB_DNS_NAME:$ADVISOR_LISTENER_PORT/"
echo "Give the target groups 2 to 3 minutes to report healthy (the first boot installs Node and your app)."
```

Notes worth reading before you run it:

- **`ensure_role_and_profile`, `ensure_target_group`, `ensure_listener`, `ensure_launch_template` and `ensure_asg`** are each written once and called twice, once per tier. When a third tier is needed, you add two lines, not a hundred.
- **Log lines in helper functions go to stderr** (`>&2`). A function that returns a value by printing it (like `ensure_target_group`) must not mix log messages into that output.
- **Each policy names one parameter ARN**, not `ssm:*` and not `*`. That is the **least privilege** idea from earlier labs, applied to secrets. You will prove it in Part H.
- **The app security group is shared by both tiers**, and the database rule allows that whole group. So an advisor instance can *reach* the database port, even though it cannot read the password. A stronger design gives each tier its own security group. See the review questions.
- **The generated user-data stays under the 16 KB limit** for EC2 user-data. The weather tier is about 13 KB with its app code embedded, so there is little room left. If you add a lot of code to an app, this limit is the first thing to check.

F2. `delete_weather_stack.sh`

Create `delete_weather_stack.sh`:

```
#!/bin/bash
set -uo pipefail

export $(grep -v '^#' .env | xargs)
export AWS_DEFAULT_REGION="$AWS_REGION"
[ -f stack_outputs.env ] && export $(grep -v '^#' stack_outputs.env | xargs)

echo "== Deleting Auto Scaling Groups (this also terminates their instances) =="
for ASG in "$WEATHER_ASG_NAME" "$ADVISOR_ASG_NAME"; do
  aws autoscaling delete-auto-scaling-group --auto-scaling-group-name "$ASG" --force-delete 2>/dev/null || true
done
for ASG in "$WEATHER_ASG_NAME" "$ADVISOR_ASG_NAME"; do
  for i in $(seq 1 20); do
```

```
    EXISTING=$(aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names "$ASG" \
      --query "AutoScalingGroups[0].AutoScalingGroupName" --output text 2>/dev/null || true)
    if [ -z "$EXISTING" ] || [ "$EXISTING" = "None" ]; then
      echo "$ASG deleted."
      break
    fi
    sleep 15
  done
done

echo "== Deleting the load balancer and target groups =="
ALB_ARN=$(aws elbv2 describe-load-balancers --names "$ALB_NAME" \
  --query "LoadBalancers[0].LoadBalancerArn" --output text 2>/dev/null || true)
if [ -n "$ALB_ARN" ] && [ "$ALB_ARN" != "None" ]; then
  aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN" || true
  aws elbv2 wait load-balancers-deleted --load-balancer-arns "$ALB_ARN" 2>/dev/null || true
fi
for TG in "$WEATHER_TG_NAME" "$ADVISOR_TG_NAME"; do
  TG_ARN=$(aws elbv2 describe-target-groups --names "$TG" \
    --query "TargetGroups[0].TargetGroupArn" --output text 2>/dev/null || true)
  if [ -n "$TG_ARN" ] && [ "$TG_ARN" != "None" ]; then
    aws elbv2 delete-target-group --target-group-arn "$TG_ARN" || true
  fi
done

echo "== Restoring the app security group and deleting the ALB security group =="
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text)
ALB_SG=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$ALB_SG_NAME" "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text 2>/dev/null || true)
if [ -n "$ALB_SG" ] && [ "$ALB_SG" != "None" ]; then
  aws ec2 revoke-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
    --protocol tcp --port 3000 --source-group "$ALB_SG" >/dev/null 2>&1 || true
fi
aws ec2 authorize-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --cidr 0.0.0.0/0 >/dev/null 2>&1 || true
if [ -n "$ALB_SG" ] && [ "$ALB_SG" != "None" ]; then
  for i in $(seq 1 10); do
    if aws ec2 delete-security-group --group-id "$ALB_SG" 2>/dev/null; then
      echo "ALB security group deleted."
      break
    fi
    echo "ALB network interfaces still releasing, waiting 15s..."
    sleep 15
  done
fi

echo "== Deleting launch templates =="
aws ec2 delete-launch-template --launch-template-name "$WEATHER_LT_NAME" >/dev/null 2>&1 || true
aws ec2 delete-launch-template --launch-template-name "$ADVISOR_LT_NAME" >/dev/null 2>&1 || true

echo "== Removing IAM instance profiles and roles =="
for PAIR in "$WEATHER_ROLE_NAME:$WEATHER_PROFILE_NAME" "$ADVISOR_ROLE_NAME:$ADVISOR_PROFILE_NAME"; do
  ROLE="${PAIR%%:*}"; PROFILE="${PAIR#*:}"
  aws iam remove-role-from-instance-profile --instance-profile-name "$PROFILE" --role-name "$ROLE" 2>/dev/null || true
  aws iam delete-instance-profile --instance-profile-name "$PROFILE" 2>/dev/null || true
  aws iam delete-role-policy --role-name "$ROLE" --policy-name "${ROLE}-policy" 2>/dev/null || true
  aws iam delete-role --role-name "$ROLE" 2>/dev/null || true
done

rm -f stack_outputs.env
echo "Weather stack removed. Your Gemini API key parameter was left in place (the last step of the experiments shows how to delete it)."
```

The order matters: instances (through the Auto Scaling Groups) go first, then the load balancer, then target groups, then the ALB security group (it cannot be deleted while the ALB's network interfaces still use it), and finally roles. **The Gemini key parameter is deliberately left alone**, so you can rebuild the stack without creating a key again. The last experiment step shows how to delete it. **The database is not touched by this script.** Delete it with `delete_database.sh` afterward.

---

13. Part G: Deploy and Test the Application

G1. Build the stack

The database should be available by now. Build the stack:

```
./create_weather_stack.sh
```

The script ends by printing the URLs. Targets need two to three minutes to report healthy while each instance installs Node.js, the database client and your app. Watch them:

```
export $(grep -v '^#' .env | xargs); export AWS_DEFAULT_REGION="$AWS_REGION"
source stack_outputs.env; source db_outputs.env
for TG in "$WEATHER_TG_ARN" "$ADVISOR_TG_ARN"; do
  aws elbv2 describe-target-health --target-group-arn "$TG" \
    --query "TargetHealthDescriptions[].TargetHealth.State" --output text
done
```

Wait until both print `healthy`.

G2. Try it by hand

```
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -m json.tool
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -m json.tool
curl -s "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" | python3 -m json.tool
```

Compare the `cache` field in the first two answers: the first is a `miss` and the second a `hit`. Open `http://<ALB DNS name>:8081/` in a browser, enter a zip code and read the suggestion. Then try `abc` and `00000`, and note the difference between the two error messages.

G3. The smoke test

A smoke test checks that the main paths work at all. It calls both tiers through the load balancer and checks the status codes, the cache, the saved history, the error handling and whether the advice came from Gemini rather than the fallback.

Create `testing/app_smoke_test.py`:

```
#!/usr/bin/env python3
"""Call both tiers through the load balancer and check the answers."""
import argparse
import json
import sys
import time
import urllib.error
import urllib.request
from pathlib import Path


def load_dotenv(path="../.env"):
    """Read KEY=VALUE lines from a .env style file into a dict."""
    values = {}
    for line in Path(path).read_text().splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, _, value = line.partition("=")
        values[key.strip()] = value.strip().strip('"').strip("'")
    return values


def get(url, timeout=30):
    """Return (status, parsed JSON or text). HTTP errors are returned, not raised."""
    try:
        with urllib.request.urlopen(url, timeout=timeout) as resp:
            body, status = resp.read().decode(), resp.status
    except urllib.error.HTTPError as err:
        body, status = err.read().decode(), err.code
    except (urllib.error.URLError, TimeoutError) as err:
        return 0, {"error": f"no connection: {err}"}
    try:
        return status, json.loads(body)
    except ValueError:
        return status, body


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--zip", default="60616", help="zip code for the happy-path checks")
    args = parser.parse_args()

    env = load_dotenv()
    alb = load_dotenv("../stack_outputs.env")["ALB_DNS_NAME"]
    weather = f"http://{alb}:{env['WEATHER_LISTENER_PORT']}"
    advisor = f"http://{alb}:{env['ADVISOR_LISTENER_PORT']}"

    results = []

    def check(name, ok, detail=""):
        results.append(ok)
        print(f"{'PASS' if ok else 'FAIL'}  {name}  {detail}")

    status, body = get(f"{weather}/weather?zip={args.zip}")
    check("weather tier returns a forecast", status == 200 and "forecast" in body, f"status {status}")

    status, body = get(f"{weather}/weather?zip={args.zip}")
    check("a repeat call is answered from the database (cache hit)",
          status == 200 and body.get("cache") == "hit", f"cache={body.get('cache') if isinstance(body, dict) else body}")

    status, body = get(f"{weather}/history")
    check("history lists the saved lookups", status == 200 and body.get("count", 0) >= 1,
          f"status {status}, count={body.get('count') if isinstance(body, dict) else '?'}")

    status, body = get(f"{weather}/weather?zip=abc")
    check("weather tier rejects a bad zip with 400", status == 400, f"status {status}")

    status, body = get(f"{weather}/weather?zip=00000")
    check("weather tier returns 404 for an unknown zip", status == 404, f"status {status}")
```

```
    t0 = time.time()
    status, body = get(f"{advisor}/advice?zip={args.zip}")
    ok = status == 200 and bool(body.get("advice"))
    check("advisor tier returns clothing advice", ok, f"status {status}, {time.time() - t0:.1f}s")
    if ok:
        print(f"      source={body.get('source')} model={body.get('model')}")
        print(f"      advice: {body['advice']}")
        check("advice came from Gemini (not the fallback)", body.get("source") == "gemini",
              "fallback means the key, quota or model name needs a look")

    status, body = get(f"{advisor}/advice?zip=abc")
    check("advisor tier rejects a bad zip with 400", status == 400, f"status {status}")

    status, body = get(f"{advisor}/")
    check("advisor home page loads", status == 200 and "<form" in str(body), f"status {status}")

    print(f"\n{sum(results)} of {len(results)} checks passed")
    sys.exit(0 if all(results) else 1)


if __name__ == "__main__":
    main()
```

Run it:

```
cd testing
python3 app_smoke_test.py
cd ..
```

All nine checks should pass. If **only** the Gemini check fails, the stack is fine and the key, the model name or the free-tier quota needs attention. If the two database checks fail, read the weather journal (Step 1 of the experiments). The troubleshooting table in Section 15.2 covers both.

✅ **Checkpoint:** The smoke test passes, and you can explain why the advisor's home page returns `200` even when the weather service is down.

---

14. Part H: Experiments

Run everything from inside `lab06/` unless a step says otherwise. Load the settings first:

```
export $(grep -v '^#' .env | xargs); export AWS_DEFAULT_REGION="$AWS_REGION"
source stack_outputs.env; source db_outputs.env
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

Some steps read an instance's logs over SSH. The security group from lab02 allows SSH from your address, and the instances carry a `Name` tag you can search for. This helper finds an instance's public address:

```
instance_ip() {   # usage: instance_ip weather-app  or  instance_ip advisor-app
  aws ec2 describe-instances \
    --filters "Name=tag:Name,Values=$1" "Name=instance-state-name,Values=running" \
    --query "Reservations[0].Instances[0].PublicIpAddress" --output text
}
```

Step 1. Read a service's log

Make a few requests, then read the weather service's journal on the instance:

```
for Z in 60616 10001 90210 60616; do curl -s -o /dev/null "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=$Z"; done
ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip weather-app)" "sudo journalctl -u weather -n 12 --no-pager"
```

Look at the `weather_success` lines. Each has the zip, the cache result (`hit` or `miss`) and how many milliseconds the request took. Compare the time of a `hit` with a `miss`. Because each line is JSON, a tool could filter on any field.

Step 2. Look at the stored data

The saved lookups are visible through the service:

```
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history" | python3 -m json.tool
```

Now look at the table itself. The database has no public address, so you query it **from the weather instance**, which has the `psql` client, the certificate bundle, and permission to read the password. The password is read straight into a variable on the instance and is never shown.

```
ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip weather-app)" \
  "PGPASSWORD=\$(aws ssm get-parameter --name $DB_PASSWORD_PARAM --with-decryption --query Parameter.Value --output text --region $AWS_REGION
   psql 'host=$DB_HOST dbname=$DB_NAME user=$DB_USER sslmode=verify-full sslrootcert=/opt/app/rds-ca.pem' \
   -c 'SELECT id, zip, created_at FROM lookups ORDER BY id DESC LIMIT 5' \
   -c 'SELECT count(*) AS saved_lookups FROM lookups'"
```

`sslmode=verify-full` makes `psql` do the same certificate check the app does. Write down how many rows there are, and why there are fewer rows than requests you made.

Step 3. Prove the cache saves work

Pick a zip you have not used yet. Ask for it twice and compare:

```
for i in 1 2; do
```

```
  curl -s -o /dev/null -w "request $i: %{http_code} in %{time_total}s  " "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=98101"
  curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=98101" | python3 -c "import sys,json; print('then cache =',
json.load(sys.stdin)['cache'])"
done
```

The miss makes two upstream API calls and a database write. The hit makes none. Write one sentence on why a short cache matters more for a service that calls a rate-limited upstream, and one on why `CACHE_SECONDS` should not be very long for weather.

Step 4. Prove the database has no backups and no public address

```
aws rds describe-db-instances --db-instance-identifier "$DB_INSTANCE_ID" \
  --query "DBInstances[0].
{Class:DBInstanceClass,Engine:Engine,MultiAZ:MultiAZ,Public:PubliclyAccessible,Encrypted:StorageEncrypted,BackupRetentionDays:BackupRetentionPeriod}"
 \
  --output table
aws rds describe-db-snapshots --query "DBSnapshots[].DBSnapshotIdentifier"
aws rds describe-db-instance-automated-backups --query "DBInstanceAutomatedBackups[].DBInstanceIdentifier"
```

You should see `BackupRetentionDays` of `0`, `Public` of `False`, `MultiAZ` of `False`, and **empty lists** for snapshots and automated backups. Then try to reach the database from your own computer:

```
timeout 6 bash -c "echo > /dev/tcp/$DB_HOST/5432" && echo "reachable (not expected)" || echo "no connection (expected)"
```

The database name resolves, but the address is private, so the connection fails.

Step 5. Who can read which secret? (least privilege, proved)

IAM can tell you what a role is allowed to do without trying it. Check both roles against both secrets:

```
GEMINI_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${GEMINI_PARAM_NAME}"
DB_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${DB_PASSWORD_PARAM}"
for ROLE in "$WEATHER_ROLE_NAME" "$ADVISOR_ROLE_NAME"; do
  ROLE_ARN=$(aws iam get-role --role-name "$ROLE" --query Role.Arn --output text)
  for P in "$DB_ARN" "$GEMINI_ARN"; do
    printf "%-40s %-50s " "$ROLE" "${P##*parameter}"
    aws iam simulate-principal-policy --policy-source-arn "$ROLE_ARN" \
      --action-names ssm:GetParameter --resource-arns "$P" \
      --query "EvaluationResults[0].EvalDecision" --output text
  done
done
```

The weather role should be `allowed` for the database password and `implicitDeny` for the Gemini key, and the advisor role the other way around. Write one sentence on what an attacker who takes over the **advisor** tier gets, and what they do not get.

Step 6. Prove the instances are reachable only through the load balancer

The app security group accepts port 3000 only from the ALB's security group. Try to reach an instance directly, then through the ALB:

```
curl -m 5 -s -o /dev/null -w "direct: %{http_code}\n" "http://$(instance_ip weather-app):3000/" || echo "direct: no answer (good)"
curl -m 5 -s -o /dev/null -w "through the ALB: %{http_code}\n" "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/"
```

The direct call should time out and the ALB call should return `200`. Explain in one sentence what a security group rule that names **another security group** as its source does that a CIDR range cannot.

Step 7. Cut the weather service off from its database

A database failure must not become a weather outage. Remove the database's network rule, then restart the weather service so it has to open a new connection (open connections are not cut when a rule is removed, so the restart forces the issue):

```
aws ec2 revoke-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
  --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID"
ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip weather-app)" "sudo systemctl restart weather"
sleep 5
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,json; d=json.load(sys.stdin); print(d['c
'cache =', d['cache'])"
curl -s -o /dev/null -w "history while the database is cut off: %{http_code}\n" "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history"
```

The forecast still arrives (the upstream calls do not need the database), with `cache` of `unavailable`, and `/history` returns `503`. Read the `db_error` line in the journal. Then put the rule back and wait for the 30 second pause to end:

```
aws ec2 authorize-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
  --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID"
sleep 35
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,json; print('cache =', json.load(sys.stc
['cache'])"
```

The cache should work again. Write down why the 30 second pause exists, and what would happen to every request without it while the database is down.

Step 8. Break the Gemini key and watch the fallback

Overwrite the key with a wrong value. The advisor keeps working, but with the fallback:

```
aws ssm put-parameter --name "$GEMINI_PARAM_NAME" --type SecureString --value "not-a-real-key" --overwrite
sleep 70       # wait for the 60 second key cache to expire
for i in 1 2 3; do
```

```
  curl -s "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" \
    | python3 -c "import sys,json; d=json.load(sys.stdin); print(d['source'], '-', d['advice'])"
done
```

Every answer says `fallback`, yet nothing returned an error. Find out why in the advisor's journal:

```
ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip advisor-app)" \
  "sudo journalctl -u advisor -n 20 --no-pager | grep gemini_error"
```

You should see the HTTP status Gemini returned (400 or 403 for a rejected key). Write down one problem with a design where users never see an error and the only evidence is a log line that nobody reads.

Restore the real key (use the hidden prompt from Part C, then store it again):

```
read -rs -p "Gemini API key: " K; echo
aws ssm put-parameter --name "$GEMINI_PARAM_NAME" --type SecureString --value "$K" --overwrite
unset K
sleep 70
python3 testing/app_smoke_test.py
```

Step 9. Fail the weather tier and time the recovery

The advisor depends on the weather service. Terminate the weather instance, and watch what the advisor says and how long the Auto Scaling Group takes to bring the service back. Use a zip you have **not** asked for recently, so the answer is not a saved one.

```
VICTIM=$(aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names "$WEATHER_ASG_NAME" \
  --query "AutoScalingGroups[0].Instances[0].InstanceId" --output text)
aws autoscaling terminate-instance-in-auto-scaling-group --instance-id "$VICTIM" \
  --no-should-decrement-desired-capacity
START=$(date +%s)

# In a second terminal, ask the advisor while the weather tier is down:
#   curl -s -w "\n%{http_code}\n" "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=33101"

until [ "$(curl -s -m 5 -o /dev/null -w '%{http_code}' \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616")" = "200" ]; do sleep 10; done
echo "Weather service recovered after $(( $(date +%s) - START )) seconds"
curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,json; print('cache =', json.load(sys.stc
['cache'])"
```

Record the recovery time. During the outage the advisor should answer `502 weather_service_unavailable` (not hang, and not crash), and its own home page should stay up. After recovery, notice the cache result for `60616`. The old instance is gone, but the saved lookups live in the database, so the **new** instance can answer from data the old one saved. Write down what that tells you about where state should live in a stack where instances are replaced. Run the smoke test again to confirm both tiers are back.

Step 10. Clean up: stack first, then the database

Delete in this order. The database cannot be removed cleanly while the instances still use it.

```
./delete_weather_stack.sh
./delete_database.sh
```

The database delete takes a few minutes. When both scripts finish, delete the Gemini key parameter (you can skip this if you plan to rebuild the lab soon). **Also delete or restrict the key itself in Google AI Studio**, since the parameter is only a copy.

```
aws ssm delete-parameter --name "$GEMINI_PARAM_NAME"
```

Now prove nothing is left, and **pay special attention to the first three lines**. They are the ones that cost money if something remains.

```
aws rds describe-db-instances --query "DBInstances[].DBInstanceIdentifier"
aws rds describe-db-snapshots --query "DBSnapshots[].DBSnapshotIdentifier"
aws rds describe-db-instance-automated-backups --query "DBInstanceAutomatedBackups[].DBInstanceIdentifier"
aws rds describe-db-subnet-groups --query "DBSubnetGroups[].DBSubnetGroupName"
aws elbv2 describe-load-balancers --query "LoadBalancers[].LoadBalancerName"
aws autoscaling describe-auto-scaling-groups --query "AutoScalingGroups[].AutoScalingGroupName"
aws ec2 describe-launch-templates --query "LaunchTemplates[].LaunchTemplateName"
aws iam list-roles --query "Roles[?starts_with(RoleName, 'itmo-444-544')].RoleName"
aws ssm describe-parameters --query "Parameters[].Name"
```

Every list should be empty. If a snapshot appears, remove it with `aws rds delete-db-snapshot --db-snapshot-identifier <name>`.

---

15. Security and Privacy Review, and Troubleshooting

15.1. Review questions

Answer these in your write-up. They are the questions a reviewer would ask about this system.

Topic	What this lab did	The question to answer
Secret storage	Gemini key and database password in SSM SecureString	Where else could each leak from (shell history, process lists, logs, screenshots, Git)?
Least privilege	Each role reads one parameter	What would change if both roles had `ssm:*` on `*`?
Network exposure	App instances accept port 3000 only from the ALB, and the	Why is the ALB open to the internet but the instances and the database are not?

	database accepts 5432 only from the app security group	
Shared security group	Both tiers share one app security group, which the database rule allows	What can an advisor instance do that it should not be able to do? How would you fix it?
Database login	The app logs in as the master user	What would you create instead, and which privileges would it need (think: only `lookups`, only `SELECT` and `INSERT`)?
Encryption	Storage is encrypted at rest, and the connection uses TLS with certificate verification	What does the certificate check protect against that TLS without it does not?
SQL injection	All queries use placeholders	Show one line of the app and say what would go wrong if it built SQL by joining strings
Backups	Retention 0, no snapshots	What would you lose if this database held real data, and what would you configure instead?
Data lifecycle	The database is deleted with no snapshot	Who decides when stored data may be destroyed in a real organization?
Untrusted output	AI text is HTML-escaped on the page	What could an attacker gain if the page printed raw model output?
Data sent to a third party	Only zip code and public forecast text	What would you have to change before using a paid or school-owned Gemini account for real users?
Abuse and cost	The advisor page is public and the free tier has a quota	How could someone use up your quota, and what could you add (rate limiting, a WAF, authentication)?
Availability	Fallback answer when Gemini fails, and the app keeps working when the database fails	Why is it important that the response says `source: fallback` or `cache: unavailable`?

15.2. Troubleshooting

Symptom	Likely cause	Fix
`create_weather_stack.sh` says `db_outputs.env` was not found	The database script has not finished, or was run from another folder	Run `./create_database.sh` inside `lab06/` and let it finish
`create_weather_stack.sh` says the Gemini parameter was not found	The key was not stored, or the name differs from `GEMINI_PARAM_NAME`	Repeat Part C3 and compare names with `aws ssm describe-parameters`
`create_database.sh` seems stuck	The first create takes 5 to 10 minutes	Wait. Watch with `aws rds describe-db-instances --query "DBInstances[0].DBInstanceStatus"`
`create-db-instance` fails with a message about the instance class or free plan	Your account's free plan limits RDS to certain classes or engines	Use `db.t4g.micro` or `db.t3.micro` with PostgreSQL, single-AZ, 20 GB. Do not turn on Multi-AZ or a larger class
`DBSubnetGroupDoesNotCoverEnoughAZs`	`AZ_A` and `AZ_B` are the same, or a subnet is missing	Check `.env`. The two zones must differ, and each must have a default subnet
Targets stay `unhealthy`	The app crashed at boot, or the first boot is still installing	Wait 3 minutes. Then SSH in and run `sudo systemctl status weather` (or `advisor`) and `sudo journalctl -u weather -n 30 --no-pager`
`base64: weather-app/app.js: No such file`	The app folders are not inside `lab06/`, or the script was run from another folder	`cd lab06` and confirm `ls` shows `weather-app` and `advisor-app`
Weather answers always show `cache: unavailable`	The weather service cannot reach or log in to the database	`sudo journalctl -u weather --no-pager | grep db_error` and read the `message`. See the next rows
`db_error` with `ENOENT` and `rds-ca.pem`	The certificate bundle was not downloaded at boot	SSH in and run `curl -sS -o /opt/app/rds-ca.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem`, then `sudo systemctl restart weather`
`db_error` with `timeout` or `ETIMEDOUT`	The database security group does not allow the app security group, or the database is not ready	Check the database rule with `aws ec2 describe-security-groups --group-ids "$DB_SECURITY_GROUP_ID"`. Check the database status
`db_error` with `password authentication failed`	The SSM parameter no longer matches the database password	You deleted or overwrote `DB_PASSWORD_PARAM` while the database existed. Delete the database and recreate it with `delete_database.sh` then `create_database.sh`
`db_error` with `AccessDeniedException` from SSM	The weather role's policy names a different parameter	Compare `DB_PASSWORD_PARAM` with the policy resource in `create_weather_stack.sh`
`/history` returns 503	Database disabled or unavailable	Same as above. `database_disabled` means `DB_HOST` was empty in the service file
Weather tier returns `502 upstream_error` with stage `zip_lookup`	The zip service is down, or the instance has no internet route	Check from your own computer with the curl from Part A. The default VPC subnets have internet access
Weather tier returns `502` with stage `nws_points` or `nws_forecast`	NWS is slow or returned an error	Retry. NWS has occasional outages. The log line names the stage
Weather tier returns `404 no_nws_coverage`	The zip is outside NWS coverage	Try a mainland US zip
Advisor returns `502 weather_service_unavailable`	The weather tier is unhealthy, or `WEATHER_URL` is wrong	Check the weather target group. On the advisor instance run `systemctl cat advisor` and look at `WEATHER_URL`
Every advice says `fallback`	A Gemini problem: key, model name or quota	On the advisor instance, `sudo journalctl -u advisor --no-pager | grep gemini_error` shows the status and reason
`gemini_error` status 404	The model name does not exist or is not on the free tier	Set `GEMINI_MODEL` in `.env` to a current Flash-Lite or Flash model, run `create_weather_stack.sh` again, then replace the advisor instance
`gemini_error` status 400 or 403	Key rejected, or the API is not enabled for the key's project	Create a new key in AI Studio, store it again, wait 60 seconds
		Wait a minute. The fallback keeps answering. Do not loop

`gemini_error` status 429	Free-tier rate limit	requests
Advisor logs an SSM `AccessDenied` on `GetParameter`	The inline policy names a different parameter ARN	Compare `GEMINI_PARAM_NAME` with the policy resource. A parameter name that starts with `/` gives the ARN `parameter/itmo-...` with a single slash
`ssh` times out	Your address changed, so the lab02 SSH rule no longer matches	Update the SSH rule on the app security group to your current address
User-data too large	Added code pushed it over 16 KB	Keep the apps small. Check `wc -c` on the generated `user-data-*.sh` before the script deletes it
`delete_weather_stack.sh` cannot delete the ALB security group	The ALB's network interfaces are still releasing	The script retries for about 2 minutes. If it still fails, run it again
`delete_database.sh` cannot delete the DB subnet group or security group	The instance is still being deleted	Wait for `describe-db-instances` to show nothing, then run the script again
A snapshot or automated backup shows up after deletion	The database was deleted another way (for example the console default)	`aws rds delete-db-snapshot --db-snapshot-identifier <name>`, and delete any retained automated backup in the console under Automated backups

---

16. Submission Checklists

16.1. Everyone (ITMO 444 and ITMO 544)

- ☐ Output of `ls -R lab06 | head -40`, showing the layout from Part 0.
- ☐ Output of the NWS calls from Part A and one sentence on why two upstream calls are needed.
- ☐ Output of `create_database.sh`, and the Step 4 output showing retention `0`, not public, single-AZ, and empty snapshot and automated backup lists.
- ☐ Screenshot of the advisor page showing a suggestion for a zip code, and the JSON from `/advice?zip=...` for the same zip.
- ☐ Output of `app_smoke_test.py` with all nine checks passing.
- ☐ Step 1: a few lines of the weather journal, with a sentence explaining one `weather_success` line.
- ☐ Step 2: the `/history` output and the `psql` query output.
- ☐ Step 3: the miss and hit timings, and your two sentences.
- ☐ Step 5: the IAM simulator results for both roles and both secrets, and your sentence about the advisor tier.
- ☐ Step 6: the direct and through-the-ALB results, and your sentence about security group sources.
- ☐ Step 7: the output while the database was cut off, the `db_error` log line, and your sentence about the 30 second pause.
- ☐ Step 8: the fallback answers, the `gemini_error` journal lines, and your sentence about the weakness of silent fallback.
- ☐ Step 9: the recovery time you measured, what the advisor returned during the outage, and your sentence about where state should live.
- ☐ One paragraph on why the secrets are in SSM and each is read by one role, instead of in `.env` or the user-data.
- ☐ One paragraph on turning backups off: what it saved, what it cost you, and when you would never do it.
- ☐ One paragraph on what data you sent to Gemini and why the free tier makes that matter.
- ☐ Answers to the Section 15.1 review questions.
- ☐ **Output of the Step 10 verification commands showing no database instance, snapshot, automated backup, load balancer or lab role remains.**

16.2. Graduate Students Only (ITMO 544): Python Checklist

Submit this **in addition to** everything in 16.1. Every resource you create or delete in this lab must come from your Python code, not from the bash scripts or hand-typed `aws` commands.

Code

- ☐ `create_database.py` and `delete_database.py` (built on your lab05 `stack/` package) create and remove the database security group and rule, the DB subnet group, the random password parameter, and the PostgreSQL instance, and write and remove the database outputs.
- ☐ The database is created with: class `db.t4g.micro`, 20 GB, encrypted, single-AZ, **not publicly accessible**, and **`BackupRetentionPeriod=0`**. It is deleted with **`SkipFinalSnapshot=True` and `DeleteAutomatedBackups=True`**.
- ☐ `create_weather_stack.py` and `delete_weather_stack.py` create and remove everything the bash stack scripts do: roles and profiles, the two least-privilege policies (one secret each), security groups, target groups, the ALB and both listeners, both launch templates and both Auto Scaling Groups.
- ☐ The ALB is created **before** the launch templates, the advisor's user-data receives the ALB DNS name, and the weather user-data receives the database address.
- ☐ All create scripts are **idempotent** (running one twice changes nothing and does not fail) and all delete scripts tolerate resources that are already gone.
- ☐ Nothing is hard-coded: names, ports, instance class and cache time come from `.env` through your config module.
- ☐ No secret appears in any file, log line, test output, user-data or screenshot. The password is made with `secrets.token_hex`, goes only to SSM and the create call, and the Gemini key is read with `getpass`.
- ☐ Code is split into small functions with docstrings, and each AWS call that can fail on a missing or existing resource handles the specific `ClientError` code.
- ☐ A Python version of the smoke test (including the cache and history checks) that you can run from `pytest`.

Tests (run with `pytest`)

- ☐ **Unit tests with moto** (no real AWS calls): the created database has class `db.t4g.micro`, `BackupRetentionPeriod` of 0, `MultiAZ` false, `PubliclyAccessible` false and encrypted storage.
- ☐ Unit test: the database security group allows port 5432 from the app security group ID and from **no CIDR range**.
- ☐ Unit test: after create then delete, `describe_db_instances`, `describe_db_snapshots` and the password parameter are all empty.
- ☐ Unit test: the weather role's policy allows `ssm:GetParameter` on exactly the database password ARN, the advisor role's on exactly the Gemini key ARN, and neither role's policy mentions the other's.
- ☐ Unit test: the ALB-first ordering holds (the advisor user-data contains the ALB DNS name), and the weather user-data contains the database host but **never the password**.
- ☐ Unit test: the generated user-data for each tier is under 16 KB.

- [ ] Unit test: running each create function twice creates no duplicates.
- [ ] **Integration tests** (marked `integration`, run against your real stack): both target groups are healthy; `/weather?zip=60616` called twice returns `miss` then `hit`; `/history` lists the lookup; a bad zip returns `400` and an unknown zip returns `404`; `/advice` returns advice; the app instances are reachable **only** through the ALB; the database port is **not** reachable from your computer.
- [ ] Integration test: `iam.simulate_principal_policy` reports each role `allowed` for its own secret and not for the other's.
- [ ] **Chaos tests** (marked `chaos`): (1) revoke the database rule and restart the weather service, assert the forecast still returns `200` with `cache` of `unavailable`, then restore the rule. (2) Terminate the weather instance, assert the advisor returns `502` (not a hang) during the outage, and assert the weather service answers `200` again within 300 seconds.
- [ ] `pytest -m "not integration and not chaos"` passes with no AWS credentials, and the summary shows every test.

Short answers

- [ ] Half a page: what moto could not prove for you in this lab (think: real backups, real network reachability, TLS), and which integration test covers each gap.
- [ ] One paragraph comparing the Python and bash versions of the same step (for example creating the database and waiting for it): what was easier to get right, and what was easier to test.

---

17. Key Terms Glossary

- **National Weather Service API:** The free US government web API for forecasts. It needs no key, but it requires a descriptive `User-Agent` header.
- **Geocoding:** Turning a place description, such as a zip code, into latitude and longitude.
- **Two-tier application:** An application split into two services that call each other, here a data tier (weather) and a logic tier (advisor).
- **Gemini API:** Google's hosted AI model API. The free tier has rate limits and may use prompts to improve Google products.
- **API key:** A secret string that identifies the caller of an API. Treat it like a password.
- **Amazon RDS:** The AWS service that runs a managed relational database for you (here PostgreSQL), handling the server, patches and storage.
- **PostgreSQL:** An open source relational database queried with SQL.
- **DB instance class:** The size of the database server. `db.t4g.micro` is the small, free-tier size.
- **Single-AZ and Multi-AZ:** One database copy in one zone, or a standby copy in a second zone. Multi-AZ is not free.
- **Automated backup and backup retention:** RDS's daily backups and transaction logs, kept for a set number of days. A retention of 0 turns them off, which also removes point-in-time recovery.
- **Snapshot:** A saved image of a database. A manual snapshot, or the final snapshot taken at deletion by default, keeps costing storage until you delete it.
- **DB subnet group:** The list of subnets, in at least two zones, that RDS may place a database in.
- **Publicly accessible:** Whether the database gets an address reachable from the internet. Ours does not.
- **Cache and time to live (TTL):** A saved copy of a recent answer reused instead of fetching again, and how long it is considered fresh. Here `CACHE_SECONDS`.
- **Connection pool:** A small set of open database connections that requests share.
- **TLS certificate verification:** Checking that the server's certificate was issued by a trusted authority for that server, which proves who you are talking to. Encryption alone does not.
- **SQL injection:** An attack where input is mixed into SQL text and changes the query. Placeholders (`$1`) prevent it.
- **SSM Parameter Store:** An AWS service for configuration values and secrets, with free standard parameters.
- **SecureString:** A Parameter Store type that is encrypted at rest and needs permission to read.
- **Least privilege:** Giving an identity only the permissions it needs, on only the resources it needs.
- **Policy simulator:** The IAM feature that reports whether a role would be allowed to make a call, without making it.
- **Instance profile:** The wrapper that attaches an IAM role to an EC2 instance.
- **Security group source:** A security group rule can allow traffic from another security group instead of an address range.
- **Graceful degradation:** Continuing to give a reduced but useful answer when a dependency fails.
- **Fallback:** The simpler answer used when the main one is unavailable. It must be labelled as such.
- **Cross-site scripting (XSS):** An attack where untrusted text is printed into a page as markup. Escape all untrusted output, including AI output.
- **Rate limit (HTTP 429):** The service telling you to slow down.
- **Structured logging:** Writing one JSON object per log line so lines can be filtered by field.
- **Health check:** The periodic request the load balancer makes to decide whether an instance should receive traffic.
- **Recovery time:** How long it takes for a failed service to answer correctly again.
- **Stateless instance:** An instance that keeps nothing important on its own disk, so it can be replaced at any time. State belongs in a database or other managed service.

---

18. References

- National Weather Service API documentation: https://www.weather.gov/documentation/services-web-api
- Zippopotam.us zip code API: https://api.zippopotam.us
- Google AI Studio and the Gemini API documentation (models, rate limits and data use terms): https://ai.google.dev/gemini-api/docs
- Amazon RDS free tier: https://aws.amazon.com/rds/free/
- Amazon RDS for PostgreSQL, working with backups, and deleting a DB instance: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
- Using SSL/TLS with RDS and the RDS certificate bundle: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.SSL.html
- AWS Systems Manager Parameter Store: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html
- IAM policy simulator: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_testing-policies.html
- OWASP guidance on cross-site scripting prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP SQL injection prevention: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html